

Handling Multimodal Documents

Your RAG pipeline reads the text and throws away the chart that contained the answer.



Where The Specifics Live

Text carries narrative. Figures and tables carry the facts people ask about.

CHARTS

The actual numbers, trends and comparisons

TABLES

Pricing, specifications, parameters, limits

DIAGRAMS

Architecture, process flow, relationships

SCREENSHOTS

UI documentation: "where is that setting?"

EQUATIONS

The formula itself, not a description of it

SCANNED PARTS

Signatures, stamps, handwritten annotations

PHOTOS

Product images, defect examples, medical imagery



"What does the Pro plan cost?" is a table question. "How does auth work?" is a diagram question. Missing those, the system should say "I do not know", but it hallucinates instead, because the nearby text sounds relevant.



Three Strategies, Three Cost Curves

Pick before you build, they are very different systems

STRATEGY	WHAT IT COSTS YOU
1. IGNORE	Index text only and drop the rest. Cost zero, quality is whatever was lost. Honest use: text heavy corpora where figures are decorative.
2. DESCRIBE TO TEXT	A vision model writes a description of each figure and you index the text. One call per figure, once. Works with your existing text pipeline.
3. NATIVE MULTIMODAL	Embed images and text in a shared space, or embed page images directly. Different infra, much larger indexes. Best for visually dense documents.



Strategy 2 is where most production systems land, with a fallback to strategy 3 for document types where layout carries meaning: slide decks, forms, engineering drawings.



Turn Figures Into Indexable Prose

The describer needs the document around the image, not just the image

```
def describe_figure(image_bytes, ctx) → str:
    prompt = f"""Describe this figure from a technical document
    so search can find it later.
```

```
Document: {ctx['doc_title']}
Section: {ctx['heading_path']}
Caption: {ctx['caption'] or '(none)'}
Text before the figure: {ctx['preceding_text'][:500]}
```

Include: the figure type, every concrete value, label, axis
and legend entry you can read, the relationship it shows,
and any text visible inside the figure.

Do not speculate. Do not start with "This image shows"."""

```
return vlm.invoke(prompt, image=image_bytes)
```

✗ WITHOUT CONTEXT

"A line chart with an upward trend,
with blue and orange lines."

Matches nothing a user would type

✓ WITH CONTEXT

"Quarterly revenue by region. EMEA
rises from 4.2M in Q1 2023 to 6.8M in
Q4 2024. APAC rises from 3.3M to
5.1M."

Retrievable by dozens of real queries



The Caption Is Your Best Signal

An image on its own is nearly meaningless to a describer

```
def gather_figure_context(page, bbox, meta) → dict:
    blocks = page.get_text("blocks")
    above = [b for b in blocks if b[3] ≤ bbox[1]]
    below = sorted([b for b in blocks if b[1] ≥ bbox[3]],
                   key=lambda b: b[1])

    # Captions sit just below and start with a marker
    CAP = r"^(figure|fig\.|table|exhibit|chart)\s*\d+"
    caption = next((b[4].strip() for b in below[:2]
                    if re.match(CAP, b[4], re.I)), None)

    prev = sorted(above, key=lambda b: -b[3])[:3]
    return {"doc_title": meta["title"], "page": page.number,
            "heading_path": meta["heading_path"],
            "caption": caption, "preceding_text": text_of(prev)}
```



ALWAYS DO THIS WITH A CAPTION

- ✓ Keep it as its own metadata field
- ✓ Include it in the description prompt
- ✓ Prepend it to the indexed description
- ✓ Use it verbatim when citing the figure back

A figure with a good caption barely needs a vision model. One without it needs every scrap of surrounding context.



Extract The Data, Do Not Describe It

"An upward trend" does not answer "what was Q3 revenue?"

```
CHART_PROMPT = """This is a chart. Extract its data as JSON:
{"chart_type": "line|bar|pie|scatter", "title": "...",
 "x_axis": {"label": "...", "unit": "..."},
 "y_axis": {"label": "...", "unit": "..."},
 "series": [{"name": "...", "points": [{"x": ..., "y": ...}]}]}
Read values from axis and data labels. Use null when a value
is not readable. Never invent points that are not visible."""
```

```
def chart_to_indexable_text(c: dict) → str:
    unit = c["y_axis"].get("unit", "")
    lines = [f"{c['title']} ({c['chart_type']} chart)."]
    for s in c["series"]:
        for p in s["points"]:
            if p["y"] is not None:
                lines.append(f"{s['name']}: {p['x']} = {p['y']} {unit}")
    return "\n".join(lines)
```

NARRATIVE

"EMEA grew steadily, accelerating after Q2 2024"

TREND QUERIES

DATA POINTS

"EMEA: Q3 2024 = 6.1M" is the chunk that answers the question

LOOKUPS



Mark chart values as approximate. A model reading pixel positions off an unlabelled axis is estimating. For financial or clinical data, link the source figure and say the number came from a chart.



Describe Relationships, Not Shapes

People search for component names and connections

```
DIAGRAM_PROMPT = """This is a diagram from technical docs.
```

Describe it so an engineer searching for how this system works would find your description. Include:

- every labelled component or box, by its exact label
- every connection: what connects to what, and which way
- any labels on the arrows or edges
- conditional and branching paths
- groupings, boundaries or swimlanes, and what they mean

Write the flow as sequential statements, for example:

```
"The API Gateway forwards authenticated requests to the  
Order Service."
```

Do not describe colours, shapes or positions. Nobody searches for those."""

✗ VISUAL DESCRIPTION

"Five blue rectangles connected by black arrows, left to right, with one branch downward."

Matches no realistic query

✓ RELATIONAL DESCRIPTION

"The API Gateway validates the token with the Auth Service, then forwards to the Order Service, which writes to the Orders database. On failure it returns 401."

Matches "how does auth work", "what happens on 401", "what writes to the orders database"



Transcribe UI Text Verbatim

Users search for the literal label they see on screen

```
SCREENSHOT_PROMPT = """This is a product documentation
screenshot. Describe it for someone searching for how to do
something in this interface. Include:
- the screen or page name, if visible
- every menu, tab, button, field and toggle label, verbatim
- the navigation path implied, e.g. Settings > Billing
- any values, states or error messages shown
- what action the screenshot demonstrates
```

Transcribe UI text EXACTLY as written, capitalisation included. Never paraphrase a label."""

QUERY

"where is enable two-factor authentication"

PARAPHRASED

"A settings page with options to turn on additional login verification"

NO MATCH

VERBATIM

"Security settings. Toggle 'Enable two-factor authentication', 'Manage devices' button, 'Recovery codes' link. Path: Settings > Security"

STRONG MATCH



For UI content the **exact label text is the retrieval signal**, on keyword and vector search alike.



AI Coach John

Follow



 Repost

One Table, Three Representations

Index all three, because the queries are different shapes

Plan	Storage	Price	Support
Starter	10 GB	\$9/mo	email
Pro	100 GB	\$29/mo	email + chat
Enterprise	unlimited	custom	dedicated CSM

1

STRUCTURED

Markdown kept intact: "compare the plans", "show the pricing table"

2

ROW SENTENCES

"The Pro plan includes 100 GB, costs \$29/mo and has email and chat support"

3

SUMMARY

"Pricing comparison of Starter, Pro and Enterprise across storage, price and support"



Index all three and link them with a shared `table_id`, so any hit can pull the full table back.



Index The Text, Return The Original

The decoupling that makes multimodal RAG maintainable

```
def index_figure(fig, description: str, store):
    fid = f"fig_{fig.doc_id}_{fig.page}_{fig.index}"

    # Child: this is what gets embedded and searched
    store.upsert_chunk(text=description,
                      embedding=embedder.encode(description),
                      metadata={"type": "figure_description", "figure_id": fid,
                              "doc_id": fig.doc_id, "page": fig.page,
                              "caption": fig.caption, "figure_kind": fig.kind})

    # Parent: the original, fetched by id, never embedded
    store.put_artefact(fid, {"image_bytes": fig.image_bytes,
                           "table_markdown": fig.table_markdown,
                           "raw_data": fig.extracted_data})

def retrieve_with_originals(query, store, k=5):
    chunks = store.search(query, k=k)
    for c in chunks:
        if fid := c.metadata.get("figure_id"):
            c.artefact = store.get_artefact(fid)
    return chunks
```

SEARCH ON

The text description, written to be searchable

RETURN

The original image, real table markdown and extracted data

GAIN

Re-describe with a better model later without re-ingesting a single source document



Native Multimodal Retrieval

When describing to text is not enough

✂️ SHARED EMBEDDING SPACE (CLIP STYLE)

Images and text embed into one vector space, so a text query retrieves an image with no description step. **Good for** photo libraries and product images. **Weak on** dense text inside images, since these models capture overall visual semantics, not small detail.

📄 VISUAL DOCUMENT RETRIEVAL (COLPALI STYLE)

Embed the rendered page image directly with late interaction. No parsing, no OCR, no extraction at all, and the layout itself becomes signal. **Good for** slide decks, forms and drawings. **Costs** many vectors per page and a store that supports multi vector retrieval.

DESCRIBE

Works with existing text infra, cheap to store, but parsing errors propagate and a description is lossy

VISUAL

Nothing to parse wrong and layout survives, but the index is far larger and citing a span is harder



The middle ground: describe to text by default, and route the document types where parsing keeps failing, scanned forms and dense decks, to visual retrieval.



Now The Model Has To Read It

Retrieval found the figure, the prompt has to carry it

```
def build_multimodal_prompt(query, chunks) → list:
    out = [{"type": "text", "text": f"Question: {query}\n\n"}]

    for c in chunks:
        kind = c.metadata["type"]
        if kind == "figure_description":
            # Send the description AND the original image
            out.append({"type": "text", "text": c.text})
            out.append({"type": "image", "source": {
                "type": "base64", "media_type": "image/png",
                "data": b64(c.artefact["image_bytes"])}})
        elif kind == "table":
            # Tables go as markdown, far more reliable than an image
            out.append({"type": "text",
                "text": c.artefact["table_markdown"]})
        else:
            out.append({"type": "text", "text": c.text})
    return out
```

- TABLE** Markdown text, more reliable than a picture of it
- CHART** Image plus the extracted data JSON
- DIAGRAM** Image plus the text description
- SCREENSHOT** Image plus the verbatim UI text



Images cost real tokens. Send images for the top 1 or 2 visual chunks only and let the text description carry the rest.



Expensive At Both Ends

Once per figure at ingestion, and every image on every request



INGESTION, ONE TIME PER FIGURE

- Skip decorative images: logos, icons, dividers, spacers
- Deduplicate: the same header logo on 400 pages needs describing once
- Cache by image hash so reruns cost nothing
- Cheap vision model for plain figures, a stronger one only for charts



QUERY, EVERY REQUEST FOREVER

- Send images only for the top 1 or 2 visual results
- Send table markdown instead of table images
- Make image inclusion conditional on the query looking visual

```
def is_worth_describing(img, min_px=100, max_aspect=10) → bool:
    w, h = img.size
    if w < min_px or h < min_px: return False    # icon
    if max(w / h, h / w) > max_aspect: return False # rule
    return image_hash(img) not in DESCRIBED_CACHE
```



Fields That Make Figures Filterable

Identity, kind, provenance, retrieval aids, and the original

```
@dataclass
class FigureChunk:
    # identity and linkage
    figure_id: str; doc_id: str; page: int
    parent_section: str          # heading path it appeared in
    # what it is
    figure_kind: str             # chart/diagram/screenshot/table
    caption: str | None
    description: str             # this is what gets embedded
    # provenance and trust
    described_by: str            # which model wrote it
    described_at: datetime
    data_confidence: str         # exact/approximate/unreadable
    # retrieval aids
    contains_numbers: bool
    entity_labels: list[str]     # components, series, UI labels
    # the original
    artefact_ref: str            # pointer to image or table
```

FIGURE_KIND Filter to diagrams when someone asks "show me the architecture"

DATA_CONFIDENCE Surface "these numbers were read off a chart" in the answer

DESCRIBED_BY Re-describe everything from a weaker model, sources untouched

ENTITY_LABELS Hybrid keyword search on exact component and UI names



The Average Hides Everything

Measure retrieval separately for every content type

```
# {"question": "What was EMEA revenue in Q3 2024?",  
# "expected_figure_ids": ["fig_ar2024_12_1"],  
# "answer_lives_in": "chart", "expected_answer": "6.1M"}  
  
def evaluate_multimodal(eval_set, retriever, k=5) → dict:  
    by_kind = defaultdict(lambda: {"hits": 0, "total": 0})  
  
    for case in eval_set:  
        got = {c.metadata.get("figure_id")  
               for c in retriever.search(case["question"], k=k)}  
        kind = case["answer_lives_in"]  
        by_kind[kind]["total"] += 1  
        if set(case["expected_figure_ids"]) & got:  
            by_kind[kind]["hits"] += 1  
  
    return {k: v["hits"] / v["total"] for k, v in by_kind.items()}
```

Text		0.93
Table		0.88
Screenshot		0.79
Chart		0.41
Diagram		0.38



Overall hit rate **0.84** looks fine. Charts and diagrams are failing badly and the headline number conceals it completely.



Fixing The Weak Content Types

What to change when the breakdown shows a gap

CHARTS LOW

Switch to data extraction, index individual points as lines, keep axis labels and units verbatim, try a stronger vision model

DIAGRAMS LOW

Push the describer toward relationships, index every component label as an entity, add hybrid keyword search

TABLES LOW

Check they are not flattened on extraction, add row sentences, confirm headers are attached to values

SCREENS LOW

Enforce verbatim UI text, index the navigation path as its own field, lean on hybrid search

ALL LOW

Check figures are being detected at all. A near zero rate usually means ingestion is dropping them, not that description is poor



Run the sanity check first. **A hit rate near zero is an ingestion bug, not a prompt quality problem.**



Everything Assembled

Text, figures and tables, each on its own path

```
def ingest_multimodal_document(path, store):
    doc = parse_document(path)
    for page in doc.pages:
        # 1. Text chunks, the normal path
        for chunk in chunk_text(page.text, page.heading_path):
            store.upsert_chunk(chunk)
        # 2. Figures
        for fig in page.figures:
            if not is_worth_describing(fig.image): continue
            ctx = gather_figure_context(page, fig.bbox, doc.meta)
            if classify_figure(fig.image) == "chart":
                data = extract_chart_data(fig.image, ctx)
                description = chart_to_indexable_text(data) # approx
            else:
                description = describe_figure(fig.image, ctx)
            index_figure(fig, with_caption(ctx, description), store)
        # 3. Tables, all three forms under one shared id
        for t in page.tables:
            tid = f"tbl_{doc.id}_{page.number}_{t.index}"
            for c in [table_summary(t), *table_to_sentences(t)]:
                store.upsert_chunk(c, meta={"table_id": tid})
            store.put_artefact(tid, {"markdown": t.to_markdown()})
```



THE SHAPE OF IT

```
text      → chunk → embed → index
figures   → filter → context → describe → embed,
            and store the original alongside it
tables    → summary + row sentences + markdown
```



Common Mistakes

What goes wrong with multimodal corpora

× DESCRIPTION

- Dropping images and never noticing
- Describing with no surrounding context
- Describing charts instead of extracting data
- Describing diagrams visually, not relationally
- Paraphrasing UI text instead of transcribing
- Ignoring captions entirely

× STORAGE AND OPS

- Embedding images with a text only model
- Describing every logo, icon and divider
- Sending five images per request
- Passing tables as images, not markdown
- One overall number hiding chart failure
- Presenting estimated numbers as exact

SYMPTOM	LIKELY CAUSE
"it says see figure 2 and stops"	figures never indexed at all
"wrong numbers from charts"	description instead of extraction
"cannot find the diagram"	visual description, not relational
"cannot find a UI setting"	paraphrased instead of verbatim labels
"ingestion cost exploded"	describing icons and logos
"eval fine, users complain"	per type breakdown never checked



AI Coach John

Follow



Repost

Multimodal Ready Checklist

Before you call a corpus done

DETECTION

- ✓ Figures and tables extracted, not dropped
- ✓ Decorative images filtered by size and hash
- ✓ Repeated logos described once, not per page

STORAGE

- ✓ Descriptions embedded, originals stored
- ✓ figure_kind, caption, data_confidence set
- ✓ described_by recorded for re-description

DESCRIPTION

- ✓ Document, section and caption context included
- ✓ Captions kept as their own field
- ✓ Charts extracted, not narrated
- ✓ Diagrams described relationally
- ✓ Screenshots transcribe labels verbatim
- ✓ Descriptions cached by image hash

ANSWER TIME

- ✓ Tables sent as markdown, not images
- ✓ Images only for the top 1 or 2 results
- ✓ Chart numbers flagged as approximate

TABLES

- ✓ Markdown preserved, never flattened
- ✓ Row sentences for lookup queries
- ✓ Summary for discovery queries
- ✓ All three share one table_id

EVALUATION

- ✓ Figure and table answerable questions included
- ✓ Retrieval measured per content type
- ✓ Chart and diagram rates tracked over time



The Answer Is Often In The Figure

A text only pipeline answers from the sentence that says "see Figure 2"

DO THIS	AND YOU GET
DETECT AND FILTER	content in, decoration out
GATHER CONTEXT	descriptions that are actually searchable
EXTRACT CHART DATA	real numbers, not "an upward trend"
DESCRIBE RELATIONALLY	diagrams findable by component name
TRANSCRIBE UI VERBATIM	screenshots match what users type
THREE TABLE FORMS	lookup, comparison and discovery
PARENT CHILD STORAGE	search the text, return the original
CACHE BY IMAGE HASH	re-describe later for free
EVALUATE PER TYPE	see the failure the average hides



Text carries the narrative, figures carry the specifics people ask about. **Follow AI Coach John** for more production RAG engineering.

